



UTXJ

TUTUNAKU

*Tania Ibarra Salgado
Brandon Leon Cabrera
Ana Daniela López Neri*

Plan del Proceso de Desarrollo Web

Metodología Ágil: Scrum

Para el desarrollo de la plataforma web Tutunakun, se eligió la metodología ágil Scrum, ya que permite organizar el trabajo por etapas y realizar entregas parciales durante el avance del proyecto. Esta forma de trabajo se adapta a las necesidades del sistema, porque la plataforma incluye módulos de gamificación, ejercicios interactivos y contenido educativo que puede ir mejorando conforme se revise y pruebe.

El proyecto busca atender la falta de recursos digitales accesibles para el aprendizaje de la lengua totonaca, especialmente para las nuevas generaciones. Por eso, Scrum resulta útil, ya que permite hacer ajustes durante el desarrollo tomando en cuenta la retroalimentación de estudiantes, hablantes y usuarios que puedan probar la plataforma.

De esta manera, el producto final no se plantea como una herramienta fija, sino como una plataforma interactiva, accesible y fácil de usar.

Estructura del Plan de Sprints

El proyecto se divide en 4 sprints principales, con una duración aproximada de 2 a 3 semanas por sprint, con el objetivo de mantener un avance ordenado y constante.

Sprint 1: Bases del proyecto, diseño UI/UX e investigación de contenido

Objetivos:

Establecer las bases del proyecto, recopilar información lingüística del idioma totonaco y definir la experiencia visual de la plataforma.

Desarrollo:

En esta primera etapa se trabaja principalmente en la investigación del contenido educativo, ya que el proyecto está relacionado con la preservación cultural. También se diseña la interfaz en Figma con un enfoque responsivo y mobile-first, considerando que muchos usuarios podrían acceder desde un teléfono celular.

La inspiración visual basada en elementos culturales mexicanos, como los alebrijes, busca que la plataforma sea atractiva para los jóvenes y que tenga una identidad visual relacionada con la cultura.

Entregables principales:

- Modelado e implementación inicial de la base de datos en MySQL.
- Mockups de alta fidelidad en Figma.
- Estructura pedagógica de las primeras 20 lecciones.

Sprint 2: Arquitectura del backend y desarrollo de la API

Objetivos:

Desarrollar la lógica del servidor, el control de acceso y la persistencia de datos.

Desarrollo:

En este sprint se construye el backend de la plataforma, el cual será responsable de manejar usuarios, autenticación, progreso, ejercicios y conexión con las bases de datos. También se comienza a aplicar una arquitectura limpia, con el propósito de mantener el código organizado y facilitar futuras modificaciones.

Entregables principales:

- Sistema funcional de autenticación de usuarios.
- API REST estructurada.
- Documentación de la API con Swagger.
- Desarrollo de los 16 endpoints planeados para el consumo de datos.

Sprint 3: Maquetación frontend y sistema de gamificación

Objetivos:

Construir la interfaz interactiva de usuario y programar las mecánicas de gamificación.

Desarrollo:

En esta etapa se pasa el diseño visual a código utilizando React y TypeScript. También se conectan los endpoints del backend con los componentes del frontend para mostrar información dinámica al usuario.

La gamificación es una parte importante del proyecto, ya que ayuda a mantener la motivación del estudiante por medio de puntos de experiencia, niveles, corazones, rachas e insignias.

Entregables principales:

- Interfaz web responsiva desarrollada en React y TypeScript.
- Integración del sistema de gamificación.
- Visualización de puntos XP, niveles, corazones, rachas diarias e insignias.

Sprint 4: Módulos de ejercicios, pruebas y despliegue

Objetivos:

Desarrollar los ejercicios interactivos, realizar pruebas y preparar el despliegue de la aplicación.

Desarrollo:

En este sprint se completan los tipos de ejercicios que tendrá la plataforma. También se realizan pruebas funcionales y de usabilidad para verificar que el sistema sea comprensible y funcione correctamente.

Finalmente, se realiza el despliegue de la aplicación web para que pueda utilizarse desde cualquier dispositivo con conexión a internet, sin necesidad de instalar software adicional.

Entregables principales:

- Módulo con más de 100 ejercicios interactivos.
- Ejercicios divididos en 4 modalidades: traducción, selección múltiple, escritura y audio.
- Reportes de pruebas funcionales.
- Aplicación web desplegada en producción.

Justificación de la Arquitectura de Persistencia Políglota

Para el desarrollo de Tutunakun se decidió utilizar una arquitectura de persistencia políglota, es decir, el uso de más de una base de datos según el tipo de información que se necesita manejar.

Esta decisión se tomó porque no todos los datos del sistema tienen la misma estructura ni la misma función. Algunos datos, como usuarios, contraseñas, progreso y rachas, necesitan mayor control y consistencia. Otros datos, como ejercicios, lecciones, audios o contenido multimedia, pueden cambiar con mayor frecuencia y tener estructuras diferentes.

MySQL 8: Base de datos transaccional y de seguridad

MySQL se utiliza para almacenar la información que requiere una estructura organizada y consistente. Aquí se guardan datos como usuarios, perfiles, progreso, autenticación, rachas y elementos relacionados con la gamificación.

Al trabajar con el motor InnoDB, MySQL permite manejar transacciones, relaciones entre tablas e integridad de datos. Esto es importante porque la información del usuario debe mantenerse segura y no debe perderse ni duplicarse.

Por ejemplo, cuando un estudiante completa una lección, gana experiencia o mantiene una racha diaria, esos datos deben registrarse correctamente.

MongoDB: Base de datos para contenido dinámico

MongoDB se utiliza para almacenar contenido educativo que puede tener diferentes estructuras. En una plataforma de idiomas, no todos los ejercicios son iguales. Algunos contienen audio, otros respuestas de texto, imágenes o varias opciones de selección.

Por esta razón, MongoDB es útil, ya que permite guardar documentos flexibles sin depender de una estructura rígida como en una base de datos relacional.

Además, al guardar una lección completa como un documento, el backend puede consultar el contenido de forma más directa, lo que ayuda a que la experiencia del usuario sea más rápida.

FastAPI: Capa de conexión entre servicios

FastAPI funciona como la capa principal del backend. Su función es conectar el frontend con las bases de datos y controlar las operaciones del sistema.

Este framework permite manejar peticiones de manera eficiente. Por ejemplo, mientras se valida la identidad del usuario en MySQL, también se puede consultar el contenido de una lección en MongoDB.

De esta forma, FastAPI ayuda a que la plataforma tenga un mejor rendimiento y pueda responder correctamente cuando varios usuarios la utilicen al mismo tiempo.

Descripción de la Arquitectura Híbrida

El ecosistema de Tutunakun se organiza mediante una arquitectura híbrida, donde el frontend, el backend y las bases de datos cumplen funciones diferentes, pero trabajan en conjunto.

El usuario accede desde una interfaz web o móvil. Esta interfaz se comunica con la API desarrollada en FastAPI, la cual se encarga de procesar las solicitudes, validar usuarios, consultar información y devolver las respuestas necesarias.

La arquitectura se divide en dos flujos principales:

Plano de identidad y control: MySQL

Este plano se encarga de la información relacionada con usuarios, autenticación, perfiles, progreso y gamificación.

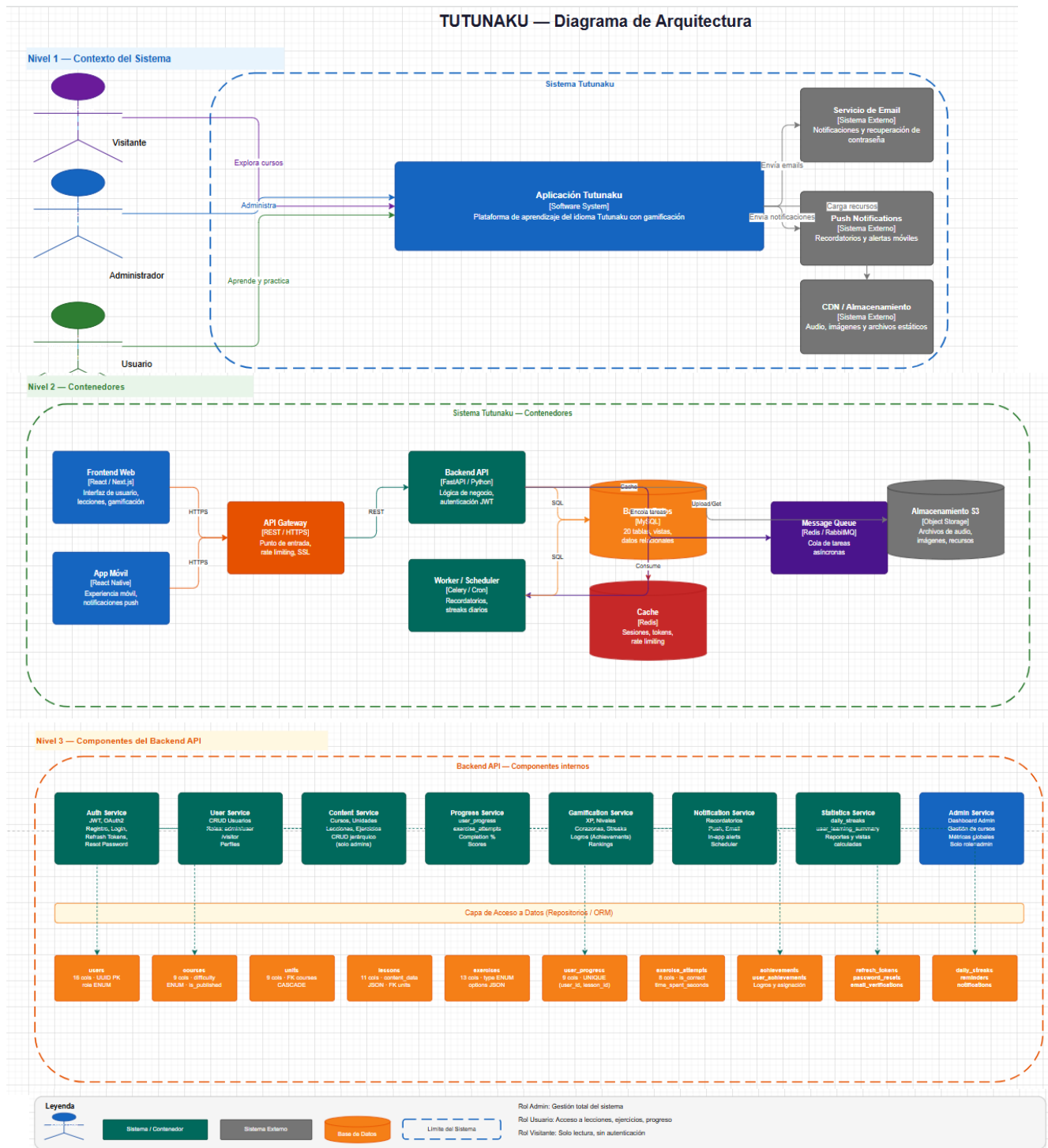
Se utiliza MySQL porque estos datos necesitan una estructura clara y relaciones bien definidas.

Plano de conocimiento y experiencia: MongoDB

Este plano administra el contenido educativo, las lecciones, los ejercicios multimedia y los intentos realizados por los estudiantes.

Se utiliza MongoDB porque este tipo de información puede variar mucho dependiendo del ejercicio o del contenido de la lección.

Diagrama de Arquitectura de Software y Modelos de Datos



En esta sección se integra el diagrama de arquitectura del sistema, donde se representa la comunicación entre el cliente web o móvil, la API desarrollada en FastAPI y las bases de datos MySQL y MongoDB.

También se muestran los modelos principales de datos utilizados para organizar usuarios, progreso, lecciones, ejercicios y contenido multimedia.

Patrones de Diseño

1. Factory Method: Motor de contenido

En una aplicación de idiomas, los ejercicios pueden tener comportamientos diferentes. Algunos reproducen audio, otros validan texto y otros muestran opciones de selección o actividades interactivas.

Si todos estos ejercicios se crearan usando demasiadas condiciones con if, else o switch, el código sería difícil de mantener y crecería de forma desordenada.

Por eso se plantea el uso del patrón **Factory Method**, ya que permite crear diferentes tipos de ejercicios desde una estructura más organizada. Así, cada tipo de ejercicio puede manejarse de forma independiente sin afectar todo el sistema.

2. Builder: Construcción de lecciones

Una lección de Tutunakun puede tener diferentes elementos. Algunas lecciones pueden incluir solo vocabulario, otras pueden tener explicación gramatical, ejercicios o actividades de repaso.

Si se usara un constructor con muchos parámetros, el código sería poco claro y complicado de modificar.

El patrón **Builder** permite construir una lección paso a paso. De esta manera, se puede crear una lección sencilla o una lección más completa usando una misma estructura, pero sin volver confuso el código.

3. Facade: Punto de control único

Cuando un usuario termina una lección, el sistema realiza varias acciones al mismo tiempo: actualiza el progreso, calcula la experiencia obtenida, reproduce una animación o sonido, revisa si el usuario subió de nivel y guarda los cambios.

Si todas estas operaciones se manejaran directamente desde el frontend, el código sería más propenso a errores.

Por eso se usa el patrón **Facade**, que permite simplificar el proceso. El desarrollador solo llama a una función principal, como finalizarSesion(), y la fachada se encarga de coordinar los demás procesos internos.

Justificación de los Frameworks de Desarrollo Web

La selección de las herramientas y frameworks del proyecto se realizó tomando en cuenta el rendimiento, la facilidad de mantenimiento, la experiencia de usuario y el enfoque mobile-first de la plataforma.

FastAPI: Backend

FastAPI se eligió para el backend porque permite crear APIs rápidas, ordenadas y fáciles de documentar. Además, genera documentación automática con Swagger, lo cual facilita las pruebas de los endpoints durante el desarrollo.

También se eligió porque está basado en Python, un lenguaje que puede facilitar futuras integraciones, como procesamiento de audio o herramientas relacionadas con lenguaje natural para apoyar el aprendizaje de la lengua totonaca.

React + Vite: Frontend

React se utiliza para construir la interfaz de usuario de manera dinámica. Esto es importante porque el sistema de gamificación necesita actualizar elementos en tiempo real, como los puntos de experiencia, corazones, rachas e insignias.

Vite se usa como herramienta de desarrollo y empaquetado, ya que permite trabajar con mayor rapidez y mejora el tiempo de carga de la aplicación final.

Esta combinación ayuda a que la plataforma sea más fluida, especialmente en dispositivos móviles.

TypeScript: Robustez en el cliente

TypeScript se integra al frontend para evitar errores comunes durante el desarrollo. A diferencia de JavaScript puro, TypeScript permite definir tipos de datos, lo que ayuda a detectar problemas antes de ejecutar la aplicación.

Esto es útil porque la plataforma maneja información compleja, como ejercicios multimedia, usuarios, progreso y respuestas de actividades.

TailwindCSS: Diseño adaptativo

TailwindCSS se utiliza para diseñar la interfaz de forma más rápida y ordenada. Al trabajar con clases de utilidad, permite crear diseños responsivos sin escribir grandes cantidades de CSS personalizado.

Esto ayuda a mantener una interfaz limpia, adaptable a diferentes pantallas y optimizada para dispositivos móviles.

MySQL: Persistencia y consistencia de datos

MySQL se usa para almacenar datos estructurados y relacionados entre sí, como usuarios, perfiles, progreso, rachas y resultados.

Su principal ventaja es que permite mantener la integridad de la información mediante relaciones, llaves primarias, llaves foráneas y transacciones.

Esto es importante para evitar pérdida o corrupción de datos dentro de la plataforma.

MongoDB: Contenido dinámico y ejercicios

MongoDB se utiliza para guardar los ejercicios y contenidos que no tienen siempre la misma estructura.

Por ejemplo, un ejercicio de audio puede tener archivos multimedia, mientras que un ejercicio de traducción puede tener textos, respuestas válidas y equivalencias.

Al ser una base de datos basada en documentos, MongoDB permite guardar esta información de manera flexible y facilita que el contenido educativo pueda crecer o modificarse con el tiempo.

Justificación de las Plataformas y Herramientas de Versionamiento

Para el proyecto Tutunakun se decidió utilizar **Git** y **GitHub** como herramientas principales de control de versiones.

Git permite llevar un historial de los cambios realizados en el código, mientras que GitHub facilita el trabajo colaborativo, la revisión de avances y la organización del proyecto.

Estas herramientas son importantes porque el sistema está dividido en frontend, backend y bases de datos, por lo que se necesita mantener un control claro de cada modificación.

Soporte para una arquitectura desacoplada

Tutunakun trabaja con un backend desarrollado en FastAPI y un frontend construido con React y TypeScript.

Git y GitHub permiten organizar estos módulos de forma separada o dentro de un mismo repositorio, dependiendo de la forma de trabajo del equipo.

Esto ayuda a evitar conflictos entre dependencias de Python y Node.js, además de permitir que cada parte del sistema avance de manera controlada.

Control de cambios en MySQL y MongoDB

El proyecto utiliza dos tipos de bases de datos: MySQL y MongoDB. Por eso, es importante versionar los scripts, modelos y esquemas que se van creando durante el desarrollo.

Git permite registrar los cambios realizados en las estructuras de datos, evitando que una modificación afecte el funcionamiento del sistema.

Esto es especialmente importante cuando el backend, el frontend y las bases de datos dependen de los mismos contratos de información.

Relación con la metodología Scrum

El desarrollo está planeado en 4 sprints de 2 a 3 semanas. GitHub ayuda a organizar el trabajo de cada sprint mediante ramas, Pull Requests y revisiones de código.

Esto permite que los avances se integren de forma ordenada y que el equipo pueda revisar qué cambios se hicieron en cada etapa.

También facilita que los asesores o integrantes del equipo puedan verificar el progreso del proyecto.

Flujo de Trabajo del Control de Versiones

Para organizar el trabajo del equipo se utiliza un flujo basado en GitHub Flow, adaptado a las necesidades del proyecto.

Este flujo permite trabajar con ramas específicas para cada tarea, integrar cambios de forma controlada y proteger las ramas principales del repositorio.

Estructura del árbol de ramas

1. main

La rama main contiene el código estable del proyecto. Esta rama representa la versión lista para producción o para una entrega final.

No se trabaja directamente sobre esta rama. Los cambios solo llegan a main después de ser revisados y aprobados.

2. develop

La rama develop funciona como la rama principal de integración. Aquí se reúnen los avances de las diferentes funcionalidades antes de pasar a producción.

Tampoco se recomienda trabajar directamente sobre esta rama, ya que debe mantenerse como un punto de integración limpio.

3. feature/

Las ramas feature/ se utilizan para desarrollar nuevas funcionalidades. Cada integrante del equipo puede crear una rama para trabajar en una tarea específica del sprint.

Ejemplos:

- feature/api-autenticacion-jwt
- feature/componente-rachas-xp
- feature/modulo-ejercicios-audio

Estas ramas se crean a partir de develop y, una vez terminadas, se integran mediante Pull Request.

4. hotfix/

Las ramas hotfix/ se utilizan para corregir errores urgentes que aparecen en producción.

Por ejemplo, si falla la conexión con MongoDB o existe un problema de seguridad con los tokens JWT, se crea una rama de corrección rápida desde main.

Después de solucionar el error, el cambio se integra tanto en main como en develop.

Parámetros de Configuración de las Herramientas de Versionamiento

Para mantener un ambiente de trabajo ordenado, se definen configuraciones básicas para Git, el archivo .gitignore y las reglas de protección en GitHub.

A. Configuración global de Git

Cada desarrollador debe configurar su nombre y correo para que los commits puedan identificarse correctamente.

```
git config --global user.name "Nombre Completo del Desarrollador"
```

```
git config --global user.email "usuario@utxj.edu.mx"
```

Esta configuración permite saber quién realizó cada cambio dentro del proyecto.

Normalización de saltos de línea

Como el equipo puede trabajar en diferentes sistemas operativos, es necesario configurar los saltos de línea para evitar conflictos innecesarios.

Para Windows:

```
git config --global core.autocrlf true
```

Para macOS o Linux:

```
git config --global core.autocrlf input
```

Esto ayuda a que Git no detecte cambios falsos por diferencias entre los formatos de salto de línea.

Editor por defecto

También se puede configurar Visual Studio Code como editor para escribir mensajes de commit más completos cuando sea necesario.

```
git config --global core.editor "code --wait"
```

B. Archivo .gitignore

El archivo .gitignore se utiliza para evitar que Git guarde archivos que no deben subirse al repositorio.

En este proyecto es importante ignorar archivos como:

- Carpetas de dependencias.
- Archivos temporales.
- Entornos virtuales.
- Archivos .env.
- Archivos de compilación.
- Credenciales o configuraciones privadas.

Esto es importante porque un archivo .env podría contener contraseñas, tokens o cadenas de conexión a bases de datos. Para evitar riesgos, se puede compartir un archivo .env.example sin información sensible.

C. Reglas de protección en GitHub

Para evitar que se suba código incompleto o con errores a las ramas principales, se configuran reglas de protección en GitHub.

1. Bloqueo de escritura directa

Se activa la opción **Require a pull request before merging**. Esto significa que ningún integrante debe subir cambios directamente a main o develop.

Todo cambio debe pasar primero por un Pull Request.

2. Revisión de código

Se establece al menos una aprobación antes de fusionar cambios.

El revisor debe verificar que el código sea claro, que cumpla con la tarea del sprint y que no afecte otras partes del sistema.

3. Validaciones automáticas

Se activa la opción **Require status checks to pass before merging**.

Esto permite ejecutar pruebas, linters o validaciones automáticas antes de aceptar un cambio.

Si alguna prueba falla, GitHub bloquea el merge hasta que el problema se corrija.

4. Historial limpio

Se utiliza una estrategia como **Squash and merge** o **Rebase** para mantener un historial más claro.

Esto permite que varios commits pequeños se unan en un solo commit más organizado al momento de integrar una funcionalidad.

Con estas configuraciones, el proyecto Tutunakun puede mantener un desarrollo más ordenado, seguro y fácil de revisar durante cada etapa del trabajo.

Enlace del Repositorio en Funcionamiento con la Estructura del Flujo de Trabajo

El proyecto **Tutunakun** cuenta con un repositorio en GitHub, donde se almacena el código fuente de la plataforma y se organiza el flujo de trabajo del equipo de desarrollo. En este repositorio se integran los módulos principales del sistema, como el backend desarrollado con FastAPI, el frontend construido con React y TypeScript, así como los archivos relacionados con la configuración del proyecto.

El repositorio se encuentra organizado mediante una estructura de ramas que permite separar el código estable, el código en desarrollo y las nuevas funcionalidades. La rama main se utiliza para mantener la versión estable del proyecto, mientras que la rama develop funciona como espacio de integración para los avances realizados durante cada sprint.

Además, para cada nueva funcionalidad se utilizan ramas de tipo feature/, las cuales permiten trabajar de forma ordenada en tareas específicas sin afectar directamente el código principal. Cuando una funcionalidad se termina, se integra mediante un Pull Request, donde se revisa el código antes de unirlo a la rama correspondiente.

La estructura del flujo de trabajo dentro del repositorio es la siguiente:

- main: contiene la versión estable del proyecto.
- develop: integra los avances del equipo antes de pasar a producción.
- feature/: se utiliza para desarrollar nuevas funcionalidades.
- hotfix/: se utiliza para corregir errores urgentes en la versión estable.

Ejemplos de ramas utilizadas en el proyecto:

- feature/api-autenticacion-jwt
- feature/componente-rachas-xp
- feature/modulo-ejercicios-audio

Enlace del repositorio:

<https://github.com/DanielaLopez66/Tutunaku-.git>

Este repositorio permite llevar un control claro de los cambios realizados durante el desarrollo del proyecto, además de facilitar la colaboración entre los integrantes del equipo y mantener una organización adecuada conforme a la metodología Scrum.

1. Principios de Codificación Segura y Mitigación de Riesgos

El desarrollo del proyecto **Tutunakun** se rige bajo el principio de *seguridad por diseño*, implementando controles estrictos en cada capa del software para mitigar las vulnerabilidades críticas descritas en estándares internacionales como el **OWASP Top 10**.

- **Validación y Sanitización de Entradas:** Se implementa una estrategia de doble barrera. En el *frontend*, se validan los formatos de datos en tiempo real para mejorar la experiencia de usuario. En el *backend*, se utiliza **Pydantic** (integrado en FastAPI) para forzar un tipado estricto, sanitizar los datos de entrada y prevenir ataques de **Inyección SQL** e **Inyección NoSQL**.
- **Criptografía y Resguardo de Secretos:** Las contraseñas de los usuarios nunca se almacenan en texto plano; se encriptan utilizando el algoritmo de hashing **bcrypt**, el cual incluye un *salt* aleatorio para resistir ataques de diccionario y tablas de encriptación (*rainbow tables*). Asimismo, los datos sensibles del sistema (llaves de APIs, credenciales de bases de datos) se gestionan fuera del código fuente mediante variables de entorno en archivos `.env`, los cuales están estrictamente excluidos del control de versiones.
- **Seguridad en el Transporte y Control de Errores:** Toda la comunicación entre el cliente y el servidor se cifra en tránsito mediante el protocolo **HTTPS (TLS/SSL)**. Adicionalmente, el sistema cuenta con un middleware global para el **manejo de excepciones**. Esto garantiza que, ante un fallo en el servidor, el usuario reciba un mensaje genérico y controlado, evitando fugas de información (*information disclosure*) que revelen detalles de la infraestructura o la base de datos a atacantes potenciales.

2. Conceptos de Web Services y Arquitectura REST

Los servicios web (*Web Services*) actúan como el tejido conectivo de la plataforma, permitiendo la interoperabilidad entre tecnologías heterogéneas mediante estándares abiertos y desacoplados.

Características del Servicio Web implementado:

- **Estilo Arquitectónico REST:** Se diseñó una API siguiendo los principios de **REST (Representational State Transfer)**, lo que significa que el sistema no genera estado entre peticiones (*stateless*) y expone los recursos de la plataforma de manera lógica y predecible.
- **Uso Eficiente de HTTP:** La API aprovecha la semántica nativa del protocolo **HTTP** utilizando sus métodos estándar para modelar las operaciones CRUD:
 - GET: Recuperación de lecciones y datos de usuario.
 - POST: Registro de nuevos usuarios o envíos de ejercicios.

- PUT / PATCH: Actualización del perfil o del progreso de gamificación.
- DELETE: Eliminación de recursos específicos.
- **Formato de Intercambio (JSON):** Se utiliza **JSON (JavaScript Object Notation)** como formato universal para el intercambio de datos debido a su ligereza, legibilidad humana y compatibilidad nativa tanto con Python (FastAPI) como con JavaScript/TypeScript (React).

1. Servicios Web Externos (Existentes)

A. MySQL Database Service (Plano de Identidad y Control)

- **Descripción:** Servicio de base de datos relacional de nivel empresarial encargado de almacenar la información estructurada que requiere absoluta consistencia transaccional. En este servicio se guardan las tablas de perfiles de usuario, el estado de la gamificación (puntos de experiencia, vidas y rachas) y el registro de accesos del sistema.
- **Implementación:** Se integra en el backend (FastAPI) de manera asíncrona mediante un mapeador objeto-relacional (ORM) que gestiona las consultas hacia la base de datos. El backend se conecta usando una dirección de enlace segura construida a partir de variables de entorno privadas.
- **Certificados de Seguridad:** Para proteger los datos de los usuarios en tránsito, se fuerza una conexión cifrada SSL/TLS. Se emplea un certificado de Autoridad Certificadora (archivo de extensión PEM) provisto por el proveedor de hosting de la base de datos, garantizando que ninguna contraseña o dato de perfil viaje expuesto por la red.

B. MongoDB Atlas (Plano de Conocimiento y Contenido Dinámico)

- **Descripción:** Servicio de base de datos NoSQL basada en documentos que almacena la estructura flexible de las lecciones de totonaco y los más de 100 ejercicios interactivos (traducción, escritura y audios). Su flexibilidad permite registrar diferentes formatos de preguntas sin alterar una estructura rígida de tablas.
- **Implementación:** El backend en FastAPI se comunica con este servicio en la nube a través de un controlador oficial asíncrono para Python. Esto permite buscar y extraer rápidamente las lecciones multimedia que solicita el usuario desde su pantalla en React.
- **Certificados de Seguridad:** MongoDB Atlas exige de manera obligatoria la encriptación de datos en tránsito mediante protocolos de seguridad TLS modernos. La validación se realiza mediante certificados digitales de confianza universal ya instalados en el servidor, reforzado por un filtrado de seguridad por lista blanca que solo permite el acceso a direcciones IP autorizadas.

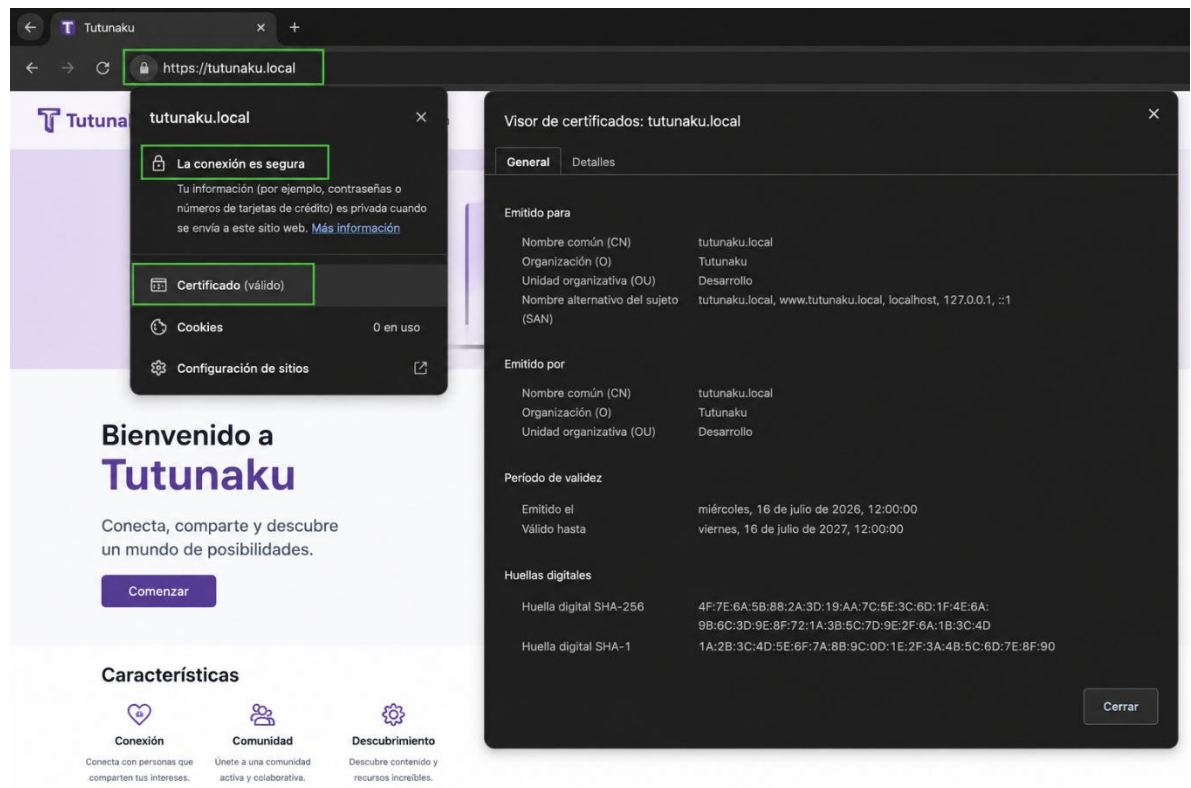
C. SendGrid / SMTP Email Service (Verificación de Cuentas, Notificaciones y Recuperación de Contraseña)

- **Descripción:** Servicio web externo especializado en la entrega y gestión de correos electrónicos transaccionales. Se encarga de enviar un correo de confirmación con un enlace o código de seguridad único para verificar y validar la identidad de los nuevos usuarios antes de permitirles el acceso completo, garantizando que el correo registrado sea real y protegiendo la plataforma de cuentas falsas o bots. También gestiona avisos importantes del sistema y el envío de enlaces seguros para restablecer contraseñas.
- **Implementación:** FastAPI se conecta a la API de este servicio de manera segura enviando una clave de autenticación privada. Al registrarse un usuario en la plataforma, el backend genera un token temporal de validación, lo asocia a su perfil y solicita a este servicio externo el envío automático de la plantilla de verificación. Una vez que el usuario hace clic en el enlace recibido, el backend cambia su estado a "usuario verificado" en la base de datos de MySQL para activar su cuenta.
- **Certificados de Seguridad:** Toda la comunicación entre nuestro backend y la plataforma de correos se cifra bajo HTTPS mediante certificados TLS estándar provistos por emisores globales de confianza, evitando la suplantación de identidad en los correos salientes y asegurando la entrega del enlace de verificación directamente a la bandeja de entrada del estudiante.

2. Servicio Web Propio (Nuevo del Proyecto)

E. Tutunaku Core REST API (El Backend en FastAPI)

- **Descripción:** Es el servicio web central desarrollado específicamente para el proyecto. Funciona como el cerebro de la plataforma, exponiendo los endpoints que procesan la lógica de la aplicación: validación de respuestas, cálculo de puntos XP, control de rachas diarias y entrega de contenido educativo almacenado.
- **Implementación:** La aplicación web interactiva en React (frontend) consume de manera asíncrona este servicio web propio. Envía solicitudes HTTPS de forma continua y adjunta un token seguro de autenticación en las cabeceras para validar la identidad de los estudiantes antes de guardar sus progresos.
- **Certificados de Seguridad:** En el entorno de producción, este servicio web se despliega bajo el protocolo seguro HTTPS utilizando un certificado de seguridad X.509 estándar emitido por la entidad certificadora abierta y gratuita Let's Encrypt. Este certificado digital se instala en el servidor web que aloja la API y cuenta con un proceso de renovación automática para mantener el canal siempre encriptado de extremo a extremo.



3. Estrategia de Consumo de APIs Propias y de Terceros

La arquitectura de Tutunaku está diseñada bajo un principio de **alta cohesión y bajo acoplamiento**, lo que permite una convivencia armónica entre los servicios core del sistema y servicios externos especializados.

Clasificación de APIs en el ecosistema:

- **API Propia (Core de la Plataforma):** Desarrollada a la medida para resolver la lógica de negocio central del proyecto. Administra de forma nativa el registro y autenticación de usuarios, el árbol de lecciones, la lógica de los ejercicios interactivos y el motor de gamificación (cálculo de puntajes, rachas y logros en el idioma totonaco).
- **Integración de APIs de Terceros:** Para evitar la "reinención de la rueda" y acelerar el desarrollo de funciones complejas, la arquitectura está preparada para conectarse con proveedores externos mediante SDKs o peticiones HTTP directas. Esto incluye la incorporación potencial de servicios de **síntesis de voz (Text-to-Speech)** y **reconocimiento de audio** para evaluar la pronunciación del usuario, servicios de traducción automatizada, y almacenamiento en la nube (ej. AWS S3 o Cloudinary) para la gestión eficiente de recursos multimedia (imágenes y audios de las lecciones).

4. Mecanismos de Autenticación y Autorización Remota

Para garantizar que los recursos protegidos del sistema solo sean accesibles por entidades legítimas, se implementa un esquema de autenticación basado en **JSON Web Tokens (JWT)** complementado con un **Control de Acceso Basado en Roles (RBAC)**.

Flujo de Trabajo del Protocolo JWT:

1. **Acreditación:** El usuario envía sus credenciales (correo y contraseña) cifradas al endpoint de inicio de sesión.
2. **Generación del Token:** Tras validar la identidad, el backend genera un JWT firmado criptográficamente con una clave secreta. Este token contiene un *payload* con datos públicos del usuario (como su ID y rol) y una fecha de expiración estricta.
3. **Persistencia en el Cliente:** El frontend recibe el token y lo almacena de forma segura (por ejemplo, en memoria o en cookies con atributos `HttpOnly` y `Secure`) para adjuntarlo automáticamente en la cabecera `Authorization: Bearer <token>` de cada petición HTTP subsiguiente.
4. **Autorización (RBAC):** El backend descodifica y verifica la firma del token en cada solicitud. Una vez validado, un sistema de middleware evalúa si el rol del usuario (ej. *Estudiante*, *Profesor*, *Administrador*) cuenta con los permisos necesarios para ejecutar la acción solicitada, bloqueando inmediatamente cualquier intento de acceso no autorizado.

5. Arquitectura del Sistema y Relación entre Componentes

La solución adopta una arquitectura multicapa desacoplada, donde cada componente de software tiene una responsabilidad única y bien definida, comunicándose a través de interfaces claras.

Ecosistema Tecnológico de Tutunakun:

Capa	Tecnología Principal	Responsabilidad Crítica
Frontend (Presentación)	React, TypeScript, TailwindCSS	Renderizado de la interfaz de usuario, gestión del estado global de la aplicación, captura de interacciones y consumo de la API REST.
Backend (Lógica de Negocio)	FastAPI (Python)	Procesamiento de reglas de negocio, enrutamiento de peticiones, validación de esquemas, seguridad y orquestación de datos.

Capa	Tecnología Principal	Responsabilidad Crítica
Persistencia Relacional	MySQL	Almacenamiento de datos transaccionales y estructurados que requieren alta integridad (usuarios, perfiles, suscripciones, compras o roles).
Persistencia NoSQL	MongoDB	Almacenamiento de datos dinámicos, semiestructurados o de rápido crecimiento (árboles de lecciones, historial de ejercicios, logs de progreso y datos de gamificación).

1. Qué es esto y para qué sirve

Este documento explica, paso a paso, cómo se convirtió la aplicación web de Tutunaku (que hasta entonces solo corría "a mano" en una máquina de desarrollo) en un stack containerizado con Docker que se puede levantar con un solo comando, en cualquier máquina o servidor, de forma reproducible.

No es solo una configuración escrita "en papel": se construyó, se levantó de verdad, se encontraron 7 bugs reales que solo aparecían al containerizar (no en desarrollo local), se corrigieron, y se validó de punta a punta contra el stack real — registro de usuarios, login, base de datos, balanceo de carga entre dos réplicas del backend, y conexión en tiempo real (Socket.IO).

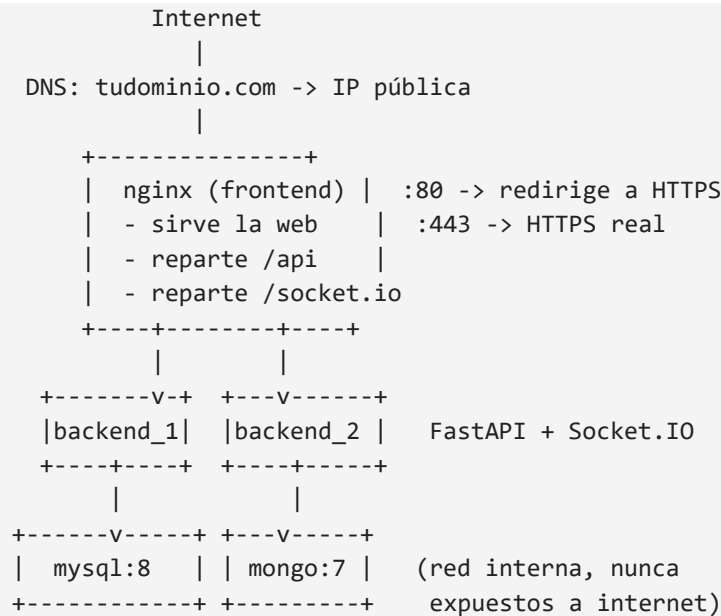
1.1 Los 5 servicios del stack

Todo corre orquestado por Docker Compose. Son 5 contenedores que trabajan juntos:

Servicio	Qué es	Se conecta con
frontend	nginx: sirve el sitio web + hace de proxy/balanceador + maneja HTTPS	Internet ⇄ backend_1/backend_2
backend_1	API (FastAPI) + tiempo real (Socket.IO), con el programador de tareas activo	mysql, mongo
backend_2	Segunda copia idéntica del backend, para repartir la carga	mysql, mongo
mysql	Base de datos relacional (usuarios, cursos, lecciones, ejercicios...)	Solo backend_1/backend_2
mongo	Base de datos de documentos	Solo backend_1/backend_2

Un sexto servicio, certbot, existe pero no se enciende con el uso normal — solo se usa una vez que hay un dominio real apuntando al servidor, para pedir el certificado de seguridad de verdad (ver sección 7).

1.2 Cómo se comunican entre sí



2. Los archivos Docker, uno por uno

2.1 backend/Dockerfile — cómo se empaqueta el servidor de la API

Convierte el código Python del backend en una imagen lista para correr, en 5 pasos:

- Parte de python:3.11-slim (una imagen ligera de Python 3.11).
- Instala las herramientas de compilación que necesitan algunas librerías de seguridad (bcrypt, argon2) para instalarse correctamente.
- Copia requirements.txt e instala todas las dependencias con pip.
- Copia el resto del código de la aplicación.
- Crea un usuario sin privilegios de administrador (tutunaku) y corre la aplicación con ese usuario, no como root — por seguridad.

El archivo completo:

```
FROM python:3.11-slim AS base

RUN apt-get update \
    && apt-get install -y --no-install-recommends build-essential libffi-dev \
```

```

    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN mkdir -p uploads \
    && useradd --create-home --uid 1000 tutunaku \
    && chown -R tutunaku:tutunaku /app
USER tutunaku

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
    CMD python -c "import urllib.request;
    urllib.request.urlopen('http://localhost:8000/health', timeout=3)" || exit 1

CMD ["uvicorn", "app.main:socket_app", "--host", "0.0.0.0", "--port", "8000", "-
    -proxy-headers", "--forwarded-allow-ips=*"]

```

Detalle importante: se arranca `socket_app` (no solo `app`) para que la API REST y el servidor de Socket.IO (usado por el dashboard en tiempo real y por el reloj Wear OS) corran juntos en el mismo proceso, exactamente igual que en desarrollo.

El HEALTHCHECK le pregunta cada 30 segundos a la propia aplicación "¿estás bien?" usando su endpoint `/health` (que ya existía y revisa que MySQL y MongoDB respondan). Docker usa esa respuesta para saber si el contenedor está realmente listo, no solo "encendido".

2.2 frontend/Dockerfile — cómo se empaqueta el sitio web

Este archivo tiene dos etapas ("multi-stage build"): primero compila el sitio (React + Vite) en una imagen con Node.js, y después copia SOLO el resultado final (archivos estáticos HTML/CSS/JS) a una imagen de nginx mucho más liviana. La imagen de Node con todas sus herramientas de desarrollo no viaja al contenedor final.

```

# --- Etapa 1: build de producción (Vite) ---
FROM node:20-slim AS build
WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY . .

```

```

ARG VITE_API_URL=/api/v1
ARG VITE_APP_NAME=Tutunaku
ENV VITE_API_URL=$VITE_API_URL
ENV VITE_APP_NAME=$VITE_APP_NAME
RUN npm run build

# --- Etapa 2: nginx sirviendo el build estático ---
FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html

HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD wget -qO- http://127.0.0.1/healthz || exit 1

```

Esta MISMA imagen de nginx no solo sirve el sitio: también hace de proxy inverso y balanceador de carga hacia el backend (ver sección 2.3) — un solo "borde" para todo, en vez de necesitar un servidor nginx aparte.

2.3 nginx: proxy inverso, balanceador de carga y HTTPS

La configuración real de nginx (deploy/nginx/default.conf.template) hace tres trabajos a la vez:

- **1.** Sirve el sitio web (los archivos que generó el build de React).
- **2.** Reparte las peticiones a /api/ y /socket.io/ entre backend_1 y backend_2 — esto es el "balanceo de carga".
- **3.** Termina la conexión HTTPS (el candado del navegador) usando un certificado de seguridad, y redirige automáticamente todo el tráfico HTTP al HTTPS.

La parte que reparte la carga:

```

upstream tutunaku_backend {
    ip_hash;
    server backend_1:8000 max_fails=3 fail_timeout=10s;
    server backend_2:8000 max_fails=3 fail_timeout=10s;
}

```

ip_hash significa "sesiones pegajosas": el mismo usuario siempre cae en la misma réplica del backend mientras dura su sesión. Esto es necesario porque la conexión en tiempo real (Socket.IO) guarda su estado en la memoria de un solo proceso — si el usuario saltara de backend_1 a backend_2 a mitad de la conexión, se rompería.

max_fails / fail_timeout: si una réplica deja de responder 3 veces seguidas, nginx deja de mandarle tráfico durante 10 segundos, para que los usuarios no vean errores.

Las rutas más importantes que reparte nginx:

Ruta	A dónde va	Por qué
/	Archivos estáticos del sitio (React)	La aplicación en sí
/api/	backend_1 o backend_2 (balanceado)	Toda la API REST
/socket.io/	backend_1 o backend_2 (mismo que la sesión)	Tiempo real (dashboard, reloj)
/uploads/	backend_1 o backend_2	Archivos subidos por usuarios (avatares...)
/healthz	Respuesta fija "ok"	Para que Docker sepa que nginx está vivo

2.4 docker-compose.yml — la orquesta que junta todo

Este es el archivo central: describe los 5 servicios, cómo se construyen, qué variables de entorno reciben, cómo se conectan entre sí, y en qué orden deben arrancar.

Orden de arranque garantizado

Docker Compose no enciende todo al mismo tiempo sin control. backend_1 y backend_2 tienen esta instrucción:

```
depends_on:
  mysql: { condition: service_healthy }
  mongo: { condition: service_healthy }
```

Es decir: "no arranques hasta que mysql y mongo hayan pasado su propio healthcheck", no solo hasta que el contenedor exista. Esto evita que el backend intente conectarse a una base de datos que todavía se está inicializando.

Cómo arranca la base de datos por primera vez

MySQL se inicializa automáticamente la primera vez (volumen vacío) cargando el esquema real y versionado del proyecto — el mismo schema.sql y seeds.sql que usa el resto del proyecto, sin scripts de migración escritos a mano:

```
volumes:
  - mysql_data:/var/lib/mysql
  - ../../../../DataBases/SQL/DD/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro
  - ../../../../DataBases/SQL/DD/seeds.sql:/docker-entrypoint-initdb.d/02-seeds.sql:ro
```

Importante: esos scripts SOLO corren la primera vez que el volumen mysql_data está vacío. Si ya existen datos, no se vuelven a ejecutar (ver la nota sobre contraseñas en la sección 5).

Las dos réplicas del backend, y por qué no son 100% idénticas

backend_1 y backend_2 usan exactamente la misma imagen y casi las mismas variables — comparten un bloque de configuración con &backend_env / <<: *backend_env (una función nativa de YAML para no repetir 20 líneas dos veces). La única diferencia real:

```
backend_1:
  environment:
    ...
    ENABLE_SCHEDULER: "true"      # <- corre las tareas periódicas

backend_2:
  environment:
    <<: *backend_env
    ENABLE_SCHEDULER: "false"     # <- NO las repite
```

Por qué: la aplicación tiene tareas que se repiten solas cada cierto tiempo (actualizar estadísticas de la plataforma, revisar el progreso de usuarios). Esas tareas viven en la memoria de un proceso. Si las dos réplicas las ejecutaran, cada tarea se dispararía por duplicado. Por eso solo backend_1 las tiene activas.

3. Variables de entorno (.env) — todas, explicadas

En este proyecto hay DOS archivos .env distintos, y es importante no confundirlos:

Archivo	Para qué es	Lo usa
backend/.env	Desarrollo local "a mano" — correr el backend directo con Python/uvicorn en tu propia máquina, sin Docker	El servidor de desarrollo nativo (venv_run)
deploy/.env	El stack completo de Docker (los 5 contenedores)	docker-compose.yml, vía env_file: .env

Ninguno de los dos se sube al repositorio (git) con datos reales — cada uno tiene su propio archivo de ejemplo (.env.example) con marcadores de posición, que se copia y se rellena con valores reales solo en la máquina donde se va a correr.

3.1 deploy/.env — variables del stack de Docker

Este es el que usa docker-compose.yml. Tabla completa de cada variable:

Variable	Qué controla	Ejemplo / valor por defecto
DOMAIN	El dominio real del sitio. Debe coincidir con el que se usa para validar peticiones en el backend (TrustedHostMiddleware).	tutunaku.mx (o tutunaku.local para pruebas locales)

MYSQL_ROOT_PASSWORD	Contraseña del usuario administrador de MySQL	(generar una fuerte)
MYSQL_DATABASE	Nombre de la base de datos	tutunakun_db
MYSQL_USER / MYSQL_PASSWORD	Usuario de la aplicación en MySQL (no es root — principio de mínimo privilegio)	tutunaku_app / (generar una fuerte)
SECRET_KEY	Clave secreta general de la aplicación (FastAPI)	32+ caracteres aleatorios
JWT_SECRET_KEY	Clave con la que se firman los tokens de sesión (JWT)	32+ caracteres aleatorios, distinta a la anterior
MAIL_USERNAME / MAIL_PASSWORD	Credenciales del servidor de correo (verificación de cuenta, recuperación de contraseña)	Las de un sandbox de Mailtrap (pruebas) o un proveedor real (producción)
MAIL_FROM	Dirección "de" con la que se envían los correos	noreply@tutunaku.mx
MAIL_PORT / MAIL_SERVER	Servidor y puerto SMTP	2525 / sandbox.smtp.mailtrap.io
MAIL_TLS / MAIL_SSL	Tipo de cifrado de la conexión de correo	true / false

Todas estas variables llegan al backend a través del bloque environment de docker-compose.yml, con valores por defecto seguros donde tiene sentido (por ejemplo \${MAIL_PORT:-2525} usa 2525 si la variable no está definida).

3.2 ¿Por qué existe ALLOWED_HOSTS_RAW y qué hace?

Esta variable no estaba en el diseño original — se agregó para resolver un bug real (bug #3 de la sección 6). Controla qué nombres de dominio/host puede aceptar el backend como válidos en la cabecera Host de cada petición (una protección de seguridad estándar contra cierto tipo de ataques).

```
ALLOWED_HOSTS_RAW: "${DOMAIN:-tutunaku.mx},*.${DOMAIN:-tutunaku.mx},localhost,127.0.0.1,*.trycloudflare.com"
```

Incluye siempre localhost y 127.0.0.1 (para que el propio chequeo de salud del contenedor no se rechace a sí mismo) y opcionalmente *.trycloudflare.com (solo si se usa un túnel de Cloudflare para pruebas — ver sección 8).

3.3 backend/.env — variables del servidor de desarrollo local

Mismo propósito que el anterior, pero para cuando el backend corre directo con Python en la máquina de desarrollo (sin contenedores) — por ejemplo, para correr las pruebas automatizadas (pytest) o depurar más rápido. Las variables son casi las mismas, con algunas diferencias porque no hay Docker de por medio:

Variable	Diferencia frente a deploy/.env
MYSQL_HOST	127.0.0.1 (la base de datos corre en la misma máquina, no en otro contenedor llamado "mysql")

DEBUG	true — activa recarga automática y detalles de error completos; nunca se usa así en producción
MONGODB_URL	Puede apuntar a una base de datos en la nube (MongoDB Atlas) en vez de una local

4. Cómo se desarrolla y se levanta el stack

4.1 Primera vez

```
cd Deliverables/WebApp/source/deploy
cp .env.example .env
# abrir .env y reemplazar los valores de ejemplo por los reales

docker compose up -d --build
```

up -d --build construye las imágenes (backend y frontend) desde cero y levanta los 5 contenedores en segundo plano. La primera vez tarda varios minutos (descarga las imágenes base, instala dependencias). Las siguientes veces es mucho más rápido porque Docker reutiliza las capas que no cambiaron.

4.2 Ver que todo esté sano

```
docker compose ps
```

Debe mostrar los 5 servicios en estado healthy:

NAME	STATUS
tutunaku-backend_1-1	Up (healthy)
tutunaku-backend_2-1	Up (healthy)
tutunaku-frontend-1	Up (healthy)
tutunaku-mongo-1	Up (healthy)
tutunaku-mysql-1	Up (healthy)

4.3 Comandos del día a día

Comando	Para qué
docker compose logs -f backend_1	Ver los logs en vivo de un servicio
docker compose restart frontend	Reiniciar un solo servicio (por ejemplo tras cambiar el certificado)
docker compose up -d --build backend_1 backend_2	Reconstruir solo el backend tras un cambio de código
docker compose down	Apagar todo (los datos en los volúmenes se conservan)
docker volume rm tutunaku_mysql_data	Borrar los datos de MySQL para que se reinicialice desde cero (con el stack apagado)

4.4 Cuándo hay que recrear el volumen de MySQL

Un detalle que causó confusión real durante el desarrollo, documentado aquí para que no se repita: los scripts de inicialización de MySQL (schema.sql, seeds.sql) SOLO corren la primera vez que el volumen está vacío. Si cambias:

- El esquema de la base de datos (agregas una columna nueva, por ejemplo), o
- La contraseña de MySQL en el .env, después de que el volumen ya existía,

...esos cambios NO se aplican solos. Hay que recrear el volumen:

```
docker compose down
docker volume rm tutunaku_mysql_data
docker compose up -d --build
```

5. Cómo se verificó que funciona (no solo que "enciende")

Levantar los contenedores sin errores no prueba que la aplicación funcione de verdad. Se hicieron pruebas reales contra el stack completo, con una base de datos real (no simulada):

```
$ curl -sk -o /dev/null -w "%{http_code} -> %{redirect_url}\n" http://localhost/
301 -> https://localhost/                # HTTP redirige a HTTPS
correctamente

$ curl -sk https://localhost/healthz
ok                                         # nginx responde

$ curl -sk -X POST https://localhost/api/v1/auth/register \
  -d '{"email":"...", "username":"...", "password":"..."}'
{"success":true,"message":"Registro exitoso...","data":{"user_id":"..."}}
                                         # INSERT real en MySQL

$ curl -sk -X POST https://localhost/api/v1/auth/login \
  -d '{"email":"x@x.com", "password":"mal"}'
HTTP 401                                # credenciales inválidas -> 401
(no 500)

$ curl -sk "https://localhost/socket.io/?EIO=4&transport=polling"
0{"sid":"...", "upgrades":["websocket"],...} # tiempo real funcionando
```

Esto confirma, contra el stack real: el registro hace una escritura de verdad en MySQL, el login valida la contraseña de verdad, los errores de la aplicación se distinguen correctamente de errores de servidor, y la conexión en tiempo real (usada por el dashboard y por el reloj Wear OS) atraviesa correctamente el proxy.

5.1 Checklist recomendado antes de cada release

- `pytest -q` en verde en el backend (o falla documentada y aceptada conscientemente).
- `npm run build` del frontend sin errores de TypeScript.
- `docker compose config -q` sin errores (valida la sintaxis antes de tocar el servidor real).
- `docker compose up -d --build` con el volumen de MySQL recreado al menos una vez por cambio de esquema.
- `docker compose ps` con los 5 servicios healthy.
- Prueba manual: redirección HTTPS, `/healthz`, registro + login reales, conexión de Socket.IO.
- Confirmar que mysql/mongo no tienen puertos publicados al host (o solo a 127.0.0.1 si es para depurar).

6. Los 7 problemas reales que se encontraron y se corrigieron

Esta es la parte más valiosa de todo el proceso: containerizar la aplicación (en vez de solo documentar cómo se "debería" hacer) sacó a la luz 7 bugs reales que eran invisibles en desarrollo local, porque ahí `DEBUG=true` escondía varios de ellos (por ejemplo, dejando que el ORM creara las tablas directo en vez de usar el esquema versionado).

Bug #1 — El backend no arrancaba: `IndexError: 6`

Causa: Una línea de configuración (`config.py`) calculaba la raíz del repositorio subiendo 6 carpetas desde el archivo — eso funciona en la máquina de desarrollo, pero dentro del contenedor el código está copiado a una profundidad de carpetas distinta, y esas 6 carpetas no existen.

Solución: Se envolvió el cálculo en un `try/except` con un valor por defecto seguro. Además, las rutas que realmente importan (modelos de aprendizaje automático, base de datos analítica)

ahora se pasan explícitas por variable de entorno, así que ese cálculo por defecto ya casi nunca se usa.

Bug #2 — nginx no arrancaba: `"host not found in upstream backend_1:8000"`

Causa: Consecuencia directa del bug 1: si `backend_1/backend_2` no llegan a arrancar, Docker no tiene registro de red para esos nombres, y nginx no puede ni siquiera arrancar su configuración.

Solución: Se resolvió solo, en cuanto se arregló el bug 1 (nginx reintentará arrancar solo).

Bug #3 — El backend se veía sano en los logs, pero /health respondía 400 Bad Request

Causa: Una protección de seguridad (TrustedHostMiddleware, activa porque DEBUG=false) tenía la lista de dominios permitidos escrita directo en el código ("tutunaku.mx" nada más). El propio chequeo de salud del contenedor llega con Host: localhost, que no estaba en esa lista, así que la petición se rechazaba.

Solución: Se creó una variable de entorno nueva (ALLOWED_HOSTS_RAW) para poder configurar esa lista sin tocar código, e incluye localhost y 127.0.0.1 por defecto.

Bug #4 — Cada 20 segundos: "attempt to write a readonly database"

Causa: Una tarea periódica registra cada predicción del sistema de recomendaciones en un archivo de base de datos, pero ese archivo se había montado en modo solo-lectura por error (se asumió, incorrectamente, que era un archivo estático).

Solución: Se quitó el modo solo-lectura de ese volumen específico (el de los modelos entrenados sí se mantiene en solo-lectura, porque ese sí es estático).

Bug #5 — El frontend corría sin errores, pero Docker lo marcaba como "unhealthy"

Causa: El chequeo de salud usaba la palabra "localhost", que dentro de ese contenedor (basado en Alpine Linux) se resuelve primero a la dirección IPv6 (::1) — pero nginx solo estaba escuchando en IPv4.

Solución: Se cambió el chequeo de salud para usar la dirección IP 127.0.0.1 directamente, sin ambigüedad.

Bug #6 — El login devolvía error 500 (error de servidor) en vez de 401 (credenciales incorrectas)

Causa: El archivo de definición de la base de datos (el que crea las tablas) no tenía una columna que sí existía en el código de la aplicación. En desarrollo esto nunca se notó porque ahí la aplicación crea las tablas automáticamente a partir del código; en

el contenedor (modo producción) se usa el archivo de definición real, que se había quedado desactualizado.

Solución: Se agregó la columna que faltaba al archivo de definición de la base de datos, en el lugar correcto.

Bug #7 — Con un volumen de MySQL recién creado, el backend a veces no arrancaba: "dependency failed to start: mysql is unhealthy"

Causa: La primera vez que MySQL inicializa una base de datos vacía (crea todas las tablas, carga los datos de ejemplo) tarda más que en un reinicio normal. El chequeo de salud no tenía un "período de gracia" para ese arranque más lento, así que Docker decidía que MySQL había fallado antes de que en realidad terminara.

Solución: Se agregó un período de gracia de 40 segundos al chequeo de salud de MySQL.

6.1 Un comportamiento que no es un bug (pero confunde)

Si cambias la contraseña de MySQL en el .env DESPUÉS de que el volumen de datos ya existía, MySQL sigue usando la contraseña anterior — vas a ver "Access denied for user". No es un error de la configuración: es cómo funciona MySQL siempre (los scripts de inicialización solo corren una vez). La solución es la de la sección 4.4: borrar el volumen y dejar que se reinicialice.

6.2 Otro bug real encontrado después: los contadores siempre en cero

Ya con el stack funcionando, se encontró otro bug real (esta vez en el código de la API, no en la configuración de Docker): los campos que cuentan cuántas lecciones tiene una unidad o cuántos ejercicios tiene una lección estaban declarados en las respuestas de la API, pero el código nunca los calculaba — siempre devolvían 0, sin importar cuántos hubiera de verdad en la base de datos. Se corrigió agregando el cálculo real (un COUNT contra la base de datos) en los tres endpoints afectados (cursos, unidades, lecciones).

7. Cómo publicarlo en un servidor real en internet

Todo lo anterior se probó en un entorno de desarrollo local, con un dominio de prueba. Para que esté disponible en internet de verdad hacen falta cosas que solo el

dueño del proyecto puede hacer (tienen costo): contratar un servidor y, si se quiere un dominio propio, comprarlo.

- 1. Crear el servidor.** Cualquier máquina virtual Linux con Docker sirve (DigitalOcean, AWS, Google Cloud, Azure, Hetzner...). Con 2 vCPU / 4 GB RAM alcanza para todo este stack.
- 2. Instalar Docker.** `curl -fsSL https://get.docker.com | sh`, más el plugin de Compose.
- 3. Apuntar el DNS.** Crear un registro A del dominio hacia la IP pública del servidor, y esperar a que se propague.
- 4. Copiar el proyecto al servidor.** Con `git clone` o `scp` — incluyendo las carpetas de modelos y base de datos analítica que están fuera de WebApp pero son necesarias.
- 5. Configurar el .env real.** Copiar `.env.example` a `.env` y reemplazar el dominio y TODAS las contraseñas/secretos por valores generados — nunca los de ejemplo.
- 6. Abrir los puertos 80 y 443.** En el firewall del proveedor de nube.
- 7. Levantar el stack (todavía sin HTTPS).** `docker compose up -d --build` — el puerto 80 necesita responder para que el siguiente paso funcione.
- 8. Pedir el certificado real.** Con el comando de Certbot (sección 2.3 / perfil tls), y reiniciar nginx.
- 9. Verificar desde fuera.** Abrir `https://tudominio.com` desde un navegador externo y repetir las pruebas de la sección 5.
- 10. Programar la renovación.** El certificado vence cada 90 días — se agrega una tarea programada (cron) que lo renueva solo.

8. Ajustes adicionales hechos durante las pruebas

8.1 Ver la base de datos con MySQL Workbench

Por diseño, MySQL no expone ningún puerto fuera de la red interna de Docker (ni siquiera a la propia máquina) — es una buena práctica de seguridad. Para poder inspeccionar los datos con una herramienta como MySQL Workbench durante el desarrollo, se publicó el puerto, pero solo hacia 127.0.0.1 (nunca hacia la red), y en el puerto 3307 en vez del 3306 estándar (porque esa máquina ya tenía otro MySQL nativo usando el 3306):

```
ports:
  - "127.0.0.1:3307:3306"    # solo en la máquina local, no en producción
```

Esto se quita por completo en un servidor de producción real — ahí la base de datos solo debe ser alcanzable desde backend_1/backend_2.

8.2 Un dominio local para pruebas: tutunaku.local

Para probar con un nombre más parecido al real (en vez de "localhost") sin poseer un dominio de verdad todavía, se generó un certificado autofirmado para tutunaku.local y se agregó al archivo hosts de Windows, apuntando a 127.0.0.1. Esto es puramente para pruebas en la máquina local — el navegador muestra una advertencia de "certificado no confiable" (normal y esperado con un certificado autofirmado).

8.3 Exponer el stack local a internet temporalmente (Cloudflare Tunnel)

Para poder probar la aplicación web y la aplicación móvil/reloj contra el mismo stack de Docker sin desplegar todavía en un servidor real, se usó un "túnel" gratuito de Cloudflare (cloudflared), que expone la máquina local con una URL HTTPS real (con certificado válido de verdad, sin advertencias) mientras el proceso sigue corriendo. Es temporal: la URL cambia cada vez que se reinicia el túnel, y deja de funcionar si se cierra el proceso o se apaga la máquina — no reemplaza un despliegue real.

8.4 Correo real de prueba (Mailtrap)

Para poder probar el flujo completo de verificación de cuenta y recuperación de contraseña sin arriesgarse a mandar correos a direcciones reales por accidente, se conectó el backend a un sandbox gratuito de Mailtrap — un servicio que captura los correos en una bandeja de entrada de pruebas en vez de entregarlos de verdad. Para producción, esto se reemplaza por un proveedor real (SendGrid, Amazon SES, u otro SMTP) simplemente cambiando las variables MAIL_* del .env — el código no cambia.

9. Notas de seguridad

- Ningún secreto real vive en el repositorio: el archivo .env.example solo tiene marcadores de posición; el .env real con contraseñas se crea directo en cada máquina y nunca se sube a git.
- MySQL y MongoDB no exponen ningún puerto fuera de la red interna de Docker en producción — solo son alcanzables desde el backend.
- El backend corre dentro del contenedor con un usuario sin privilegios de administrador, no como root.
- HSTS (Strict-Transport-Security) queda activado una vez que hay HTTPS real, forzando a los navegadores a usar siempre la conexión segura.
- En el stack de despliegue, DEBUG=false y ENVIRONMENT=production — esto desactiva la creación automática de tablas por el ORM (solo ocurre si DEBUG=true), así que en producción el

esquema de la base de datos siempre viene del archivo versionado, no de código improvisado en caliente.

10. Referencia rápida de comandos

```
# Levantar todo por primera vez
cd Deliverables/WebApp/source/deploy
cp .env.example .env          # editar con valores reales
docker compose up -d --build

# Ver estado
docker compose ps

# Ver logs de un servicio
docker compose logs -f backend_1

# Reconstruir solo el backend tras un cambio de código
docker compose up -d --build backend_1 backend_2

# Reiniciar un servicio sin reconstruirlo
docker compose restart frontend

# Apagar todo (conserva los datos)
docker compose down

# Apagar todo Y borrar los datos
docker compose down -v

# Reinicializar solo la base de datos (tras cambiar el esquema o la contraseña)
docker compose down
docker volume rm tutunaku_mysql_data
docker compose up -d --build

# Validar el archivo de configuración sin tocar nada
docker compose config -q
```